CS 527 Final Project Report: Group-15

Detecting Logic Drift in Agentic Trading-Decision Systems

Jung In Chang, Hsien-Hao Li*
University of Illinois at Urbana-Champaign
Chicago, Illinois
jichang2@illinois.edu
hl128@illinois.edu

Abstract

The integration of Agentic AI into financial decision-making introduces unique challenges regarding behavioral stability and "logic drift"—the unintended divergence of an agent's decision-making logic under volatile or adversarial inputs. While traditional validation relies on financial backtesting for profitability, such metrics often fail to capture the structural stability of the underlying decision logic, potentially masking fragility behind "lucky" profits. In this paper, we present a framework for quantifying the robustness of agentic trading-decision systems by applying three mutation operators—sentiment amplification, temporal jitter, and adversarial tweet injection—to social media sentiment signals derived from the *Cryptocurrency Tweets Dataset* [1]. We compare the behavioral stability of two agents: a deterministic rule-based policy and an LLM-based agent (*FinGPT* [9]), using regression testing on decision trajectories. Our approach introduces the Action Difference Ratio (ADR) and inter-agent consensus rate as quantitative metrics to identify susceptibility to manipulation and structural divergence between agents. We demonstrate that mutation-based analysis reveals fragility regimes that traditional backtesting ignores, and that the two agents exhibit distinct sensitivity profiles under adversarial perturbations. The entire solution can be found at: https://github.com/changju784/crypto-drift-guard.

1 Introduction

The rise of Agentic AI in high-stakes finance has shifted automated trading from static execution to autonomous decision-making. These agents process high-velocity, unstructured data—such as social media sentiment—to inform actions in volatile markets. While profitability-oriented backtesting is useful for economic evaluation, it does not directly test whether the decision logic remains behaviorally stable under controlled input perturbations.

We identify a critical research gap in software engineering (SE) regarding *logic drift*: the unintended divergence of an agent's decision logic when exposed to volatile or adversarial inputs. Unlike traditional *concept drift*, which focuses on model accuracy, logic drift addresses the *structural stability* of the decision trajectory. Current SE techniques rarely quantify how specific perturbations, such as "pump-and-dump" sentiment spikes, cause an agent to deviate from its intended risk profile.

This paper proposes a framework to bridge this gap by treating decision trajectories as regression-testable software artifacts subject to rigorous mutation and regression testing. We utilize metamorphic relations to validate behavior without a ground-truth oracle. Using 10,000 rows from the Cryptocurrency Tweets Dataset and FinBERT scoring, we evaluate a deterministic rule-based agent against a FinGPT-based LLM agent trading Bitcoin (BTC) and Ethereum (ETH). By applying mutation operators to sentiment artifacts, we measure behavioral stability through divergence metrics between baseline and perturbed trajectories.

The main contributions of this paper are:

- **Formalization of Logic Drift:** We define and distinguish logic drift from concept drift, focusing on the stability of decision structures in agentic systems.
- **Testing Architecture:** A reproducible framework for trajectory logging and automated regression testing that decouples environmental signals from decision logic.
- **Sentiment Mutation Operators:** Three operators designed to simulate adversarial environments: *sentiment amplification* (signal intensity), *temporal jitter* (timing noise), and *adversarial tweet injection* (simulated market manipulation).
- **Stability Metrics:** We introduce the *Action Difference Ratio* (ADR), *inter-agent consensus rate*, and *adversarial flip rate* to quantify behavioral stability under perturbation.

Our evaluation demonstrates that mutation testing reveals specific "fragility regimes" where decision-making becomes hypersensitive to noise. We show that rule-based and LLM-based agents exhibit distinct sensitivity profiles, providing a more granular safety metric than traditional profitability-based evaluation.

2 Proposed Technique

2.1 Approach Overview

Our technique uses a three-layered architecture to decouple environmental inputs from decision logic, enabling reproducible behavioral evaluation. By treating the trading system as a stateful software entity, we apply mutation and regression testing to detect logic drift. The framework transforms raw social media discourse into versioned artifacts, executes policies across contrasting agents, and analyzes trajectory divergence.

2.1.1 Layer 1: Sentiment Signal Generation. This layer converts unstructured data into reproducible, time-indexed artifacts using the *Cryptocurrency Tweets Dataset*.

- **Preprocessing:** Tweets are filtered by keyword (BTC/ETH) and grouped into 5-minute windows.
- **Feature Extraction:** Each tweet is scored via FinBERT (`ProsusAI/finbert`). We calculate per-window aggregates, including mean sentiment ($P(\text{pos}) - P(\text{neg})$), standard deviation, and volume.
- **Persistence:** Windowed features are saved as versioned CSVs to ensure identical, replayable inputs for all test runs.

*Project code and datasets: https://github.com/changju784/crypto-drift-guard

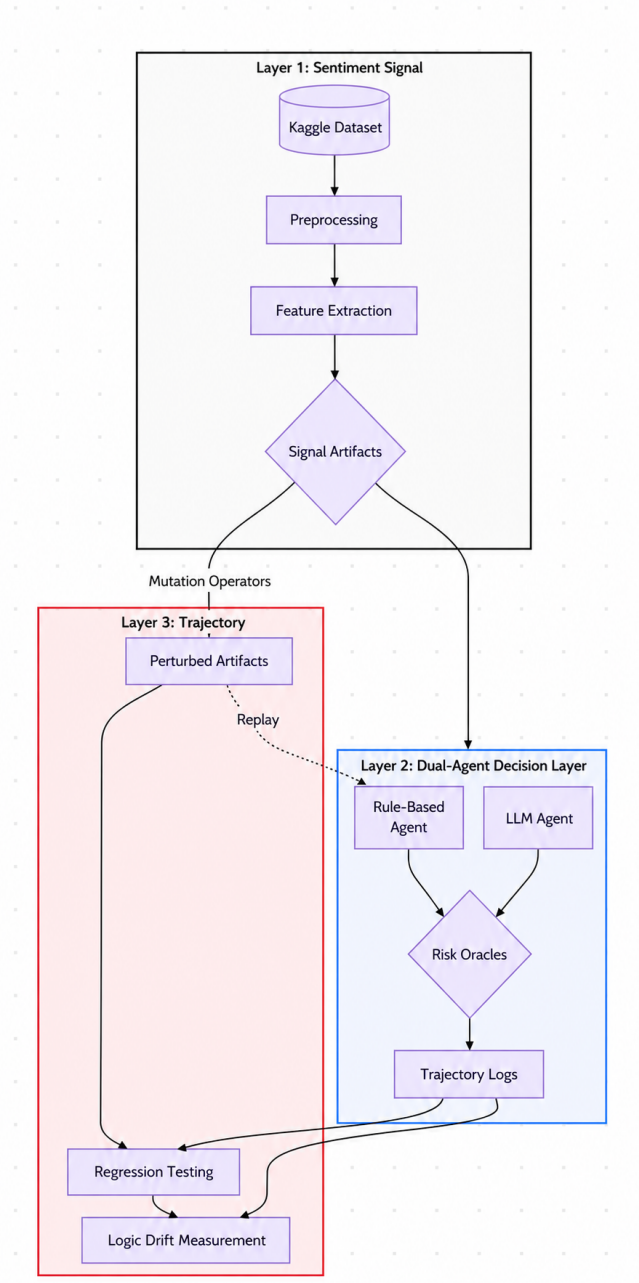

Figure 1: The three-layered architecture for logic drift detection: from data ingestion to trajectory analysis.

2.1.2 Layer 2: Dual-Agent Decision Layer. We execute trading logic across two agents to compare behavioral stability under identical inputs.

- **Rule-Based Agent:** A deterministic engine mapping features to BUY, SELL, or HOLD actions via fixed sentiment thresholds.
- **LLM-Based Agent (FinGPT):** A language model that generates trading decisions from structured text prompts containing the same window features.
- **Risk Oracle:** A shared safety layer that overrides policy to HOLD when signal quality is low (e.g., high volatility or excessive negative ratios).
- **Trajectory Logging:** The system records pre-oracle actions, final decisions, oracle triggers, and input features for post-hoc analysis.

2.1.3 Layer 3: Trajectory Analysis Layer. This layer acts as the testing suite for detecting logic drift and assessing agent robustness.

- **Mutation Testing:** We apply the three operators defined in Section 1 (amplification, jitter, and injection) to Layer 1 artifacts to simulate adversarial environments.
- **Regression Testing:** The system replays original and mutated artifacts to establish baseline and perturbed trajectories while holding all other variables constant.
- **Divergence Metrics:** We quantify drift using:
  (1) **Action Difference Ratio (ADR):** The fraction of windows where mutation changes the decision.
  (2) **Inter-agent Consensus:** The rate of agreement between the Rule-Based and LLM agents.
  (3) **Adversarial Flip Rate:** The frequency of successful decision shifts in targeted windows.

2.2 Approach Details

2.2.1 Layer 1: Sentiment Signal Generation.

Dataset and Preprocessing. We utilize a 10,000-tweet subset of the *Cryptocurrency Tweets Dataset* (Oct 2021–Nov 2022). Records are filtered for Bitcoin (BTC) and Ethereum (ETH) using regular expressions (e.g., `#btc`, `$eth`). Whitespace is normalized, but no further linguistic cleaning is applied to ensure the raw signal is preserved for scoring.

FinBERT Sentiment Scoring. Each tweet is scored via FinBERT [2], a BERT-based model pre-trained specifically on financial communications. We define the sentiment score s_i as the signed polarity:

$$s_i = P(\text{positive}_i) - P(\text{negative}_i), \quad s_i \in [-1, 1] \tag{1}$$

where $s_i > 0$ indicates bullish and $s_i < 0$ bearish sentiment. Inference is performed in batches of 64 with a 256-token limit.

Window Aggregation. Tweets are grouped into non-overlapping 5-minute windows w. For each window, we compute the feature vector:

$$\mathbf{f}_w = \left(n_w,\ \bar{s}_w,\ \sigma_w,\ r_w^+,\ r_w^0,\ r_w^-,\ \iota_w\right) \tag{2}$$

where n_w is tweet volume, $\bar{s}_w$ is mean sentiment, σ_w is standard deviation, and ι_w is mean intensity. Windows are categorized into *signal regimes* (e.g., sparse, high_conflict, neutral) to inform the adversarial targeting in M3.

Mutation Operators. To evaluate agent fragility, we introduce three mutation operators that independently transform the baseline corpus:

- **(M1) Sentiment Amplification:** Every tweet's score is scaled by $k = 1.5$, clamped to $[-1, 1]$. This simulates periods of heightened market emotion where signals are systematically stronger than historical norms, analogous to signal exaggeration observed during viral news cycles [6].

- **(M2) Temporal Jitter:** Perturbs each timestamp by $\delta_i \sim \mathcal{U}(-15, 15)$ minutes. This simulates out-of-order or delayed signal delivery arising from network latency or API rate limiting. It preserves sentiment polarity but disrupts temporal alignment, potentially splitting co-occurring bursts across window boundaries.
- **(M3) Adversarial Injection:** Simulates targeted pump-and-dump attacks [8] by injecting synthetic, high-conviction tweets into stable windows ($|\bar{s}_w| \leq 0.08$). We select up to eight high-volume near-neutral windows per coin and inject synthetic tweets (e.g., *"BTC is breaking out, buy now!"*) at a nominal ratio, with per-window synthetic counts bounded by a minimum and maximum cap. This mutation tests whether the agents and the shared oracle are sensitive to injected directional sentiment, especially high-conflict or negative-injection cases that satisfy the oracle condition.

2.2.2 Layer 2: Dual-Agent Decision Layer. Layer 2 maps the feature vectors $\mathbf{f}_w$ to discrete trading actions $a \in \{\text{BUY}, \text{SELL}, \text{HOLD}\}$. We execute a deterministic rule-based agent and an LLM-based agent in parallel. Comparing both agents across identical datasets allows us to distinguish between structural policy drift and model-specific instability.

Rule-Based Deterministic Agent. The deterministic agent follows a stateless threshold policy over mean sentiment $\bar{s}_w$:

$$\pi_{\text{det}}(\bar{s}_w) = \begin{cases} \text{BUY} & \text{if } \bar{s}_w > \theta^+ \\ \text{SELL} & \text{if } \bar{s}_w < \theta^- \\ \text{HOLD} & \text{otherwise} \end{cases} \tag{3}$$

We set $\theta^+ = 0.04$ and $\theta^- = -0.01$. These asymmetric thresholds reflect the near-neutral distribution of the baseline corpus; the conservative sell threshold prevents over-triggering on noise, while the positive buy threshold requires a clear bullish signal. This approach ensures a reproducible trajectory for regression testing.

Algorithm 1: Agent Logic with Shared Risk Oracle

Input: Window features $\mathbf{f}_w = (\bar{s}_w, \sigma_w, r_w^-, \ldots)$; Thresholds $\theta^+, \theta^-, \tau_\sigma, \tau_{r^-}$
Output: Policy action π, final action a, oracle flag o

```
// Step 1: Policy Inference
if Agent is Deterministic then
    π ← (s̄_w > θ⁺)?BUY : (s̄_w < θ⁻)?SELL : HOLD;
else if Agent is FinGPT then
    π ← LLM_Inference(Prompt(f_w));
// Step 2: Risk oracle override
if σ_w > τ_σ and r⁻_w > τ_r⁻ then
    a ← HOLD;
    o ← True;
else
    a ← π;
    o ← False;
return π, a, o;
```

LLM-Based Agent (FinGPT).. We use `bigscience/bloom-560m` [7] as the inference backbone. To avoid redundant computation, the model performs *decision inference* rather than raw text analysis, interpreting numeric window statistics directly via a structured prompt:

Prompt Generation Example:
"Instruction: Based on the crypto market sentiment data below, choose a trading action from {BUY/HOLD/SELL}:
Input: BTC: 50 tweets, mean_sentiment=0.125, positive=60%, neutral=30%, negative=10%, std=0.045
Answer: "

Greedy decoding (do_sample = False) is utilized to ensure that the LLM agent's outputs remain deterministic and reproducible for a given model state.

Shared Risk Oracle. Both agents utilize a post-inference safety oracle to suppress trades when signal quality is low. The oracle overrides the policy to HOLD if $\sigma_w > 0.30$ and $r_w^- > 0.08$ simultaneously:

$$a_w = \begin{cases} \text{HOLD} & \text{if } \sigma_w > 0.30 \wedge r_w^- > 0.08 \\ \pi_w & \text{otherwise} \end{cases} \tag{4}$$

This conjunctive condition targets high-conflict windows where signals are likely unreliable. We record the oracle trigger flag o_w separately from the policy action π_w to enable post-hoc analysis of oracle impact on trajectories.

Trajectory Logging. Each agent emits a sequence of `WindowTrajectoryEntry` records, capturing the window ID, feature vector $\mathbf{f}_w$, policy action π_w, final action a_w, and oracle status. Preserving both π_w and a_w allows Layer 3 to determine if decision changes are driven by policy drift or safety suppression.

2.2.3 Layer 3: Trajectory Analysis. Layer 3 operates on the trajectory DataFrames produced in Layer 2, computing three complementary metrics to characterize logic drift, inter-agent divergence, and adversarial susceptibility.

Action Difference Ratio (ADR). The ADR measures the frequency with which a mutation causes an agent to deviate from its baseline decision. For a baseline trajectory T^0 and a perturbed trajectory T^m over window set $\mathcal{W}$:

$$\mathcal{W} = \mathcal{W}_0 \cap \mathcal{W}_m, \quad \text{ADR}(T^0, T^m) = \frac{|\{w \in \mathcal{W} \mid a_w^0 \neq a_w^m\}|}{|\mathcal{W}|} \tag{5}$$

An ADR of 0 indicates perfect robustness, while a high ADR reveals fragility. We compute ADR for each (agent, mutation) pair and record transition counts (e.g., BUY → HOLD) to identify dominant behavioral shifts, such as oracle suppression during adversarial injections.

Algorithm 2: Layer 3 Metric Computation

Input: Trajectories $\{T^d_{\text{det}}, T^d_{\text{fgpt}}\}$ for $\mathcal{D} = \{\text{base, sentiment, jitter, adv}\}$
Output: ADR table, Consensus table (C_d), Adversarial flip rates

```
// 1. ADR: per (agent, mutation) pair
foreach agent ∈ {det, fgpt} do
    foreach m ∈ {sentiment, jitter, adv} do
        ADR ← ADR(T^base_agent, T^m_agent);
        Record (agent, m, ADR, transitions);
// 2. Consensus: per dataset
foreach d ∈ D do
    Match T^d_det and T^d_fgpt on window_id;
    C_d ← |{w : a^d_{det,w} = a^d_{fgpt,w}}| / |W^d|;
    Record (d, C_d, o_det, o_fgpt);
// 3. Adversarial flip rate: targeted windows only
foreach agent do
    W^tgt ← {w : synthetic injection > 0};
    FlipRate ← |{w ∈ W^tgt : a^base_w ≠ a^adv_w}| / |W^tgt|;
    Record (agent, FlipRate);
```

Inter-Agent Consensus Rate. Consensus (C_d) measures behavioral agreement between the deterministic and LLM agents under identical inputs:

$$C_d = \frac{|\{w \in \mathcal{W}^d \mid a^d_{\text{det},w} = a^d_{\text{fgpt},w}\}|}{|\mathcal{W}^d|} \tag{6}$$

A drop in C_d under mutation reveals structural divergence in decision logic. We track oracle triggers separately to distinguish between genuine policy agreement and forced consensus through shared safety suppression.

Adversarial Flip Rate. This metric focuses exclusively on windows $\mathcal{W}^{\text{tgt}}$ containing synthetic injections:

$$\text{FlipRate} = \frac{|\{w \in \mathcal{W}^{\text{tgt}} \mid a^{\text{base}}_w \neq a^{\text{adv}}_w\}|}{|\mathcal{W}^{\text{tgt}}|} \tag{7}$$

It quantifies susceptibility to targeted manipulation (e.g., pump-and-dump attacks [8]). We further distinguish between *targeted flips* (intended windows) and *collateral flips* (propagation to adjacent windows) to assess the breadth of the perturbation.

Parameter Sensitivity Analysis. To validate operator choices, we conducted an ablation study across three intensity levels: sentiment amplification ($k \in \{1.25, 1.50, 2.00\}$), temporal jitter ($\delta_{\max} \in \{5, 15, 30\}$ min), and adversarial injection ratios ($\rho \in \{10\%, 20\%, 35\%\}$). These curves confirm that our selected baseline parameters represent a significant but non-extreme drift regime.

3 Experimental Evaluation

3.1 Research Questions

We evaluate the behavioral stability of agentic trading systems by addressing the following questions:

- **RQ1: Agent Robustness.** Which agent architecture exhibits lower Action Difference Ratio (ADR) across mutations, and which operator drives the most significant divergence?
- **RQ2: Behavioral Consensus.** Does the agreement rate between the rule-based and LLM agents degrade under controlled input perturbations?
- **RQ3: Adversarial Susceptibility.** How effectively does the shared risk oracle suppress decision flips in targeted, near-neutral windows?
- **RQ4: Intensity Sensitivity.** How do the agents' sensitivity curves differ as perturbation intensity (scaling, jitter, and injection ratio) increases?

3.2 Experimental Setup

The evaluation utilizes the 10,000-tweet corpus (Oct 2021–Nov 2022) processed into 5-minute windowed artifacts as described in Section 2. Moreover, after BTC/ETH filtering and 5-minute aggregation, the baseline trajectory contains 64 coin-specific windows; therefore, we interpret the results as a controlled robustness case study rather than a large-scale trading benchmark. The pipeline is implemented in Python and executed via Google Colab, utilizing the *DeterministicAgent* and the *FinGPTAgent* (`bloom-560m`). All results are logged as replayable CSV artifacts to ensure experimental reproducibility.

3.3 Evaluation Plan

3.3.1 Phase 1: Baseline Establishment. We first execute both agents on unmodified window artifacts to generate reference trajectories T^0_{det} and T^0_{fgpt}. These baselines serve as regression reference trajectories for subsequent mutation comparisons, recording pre-oracle policy actions (π_w), final decisions (a_w), and oracle triggers (o_w).

3.3.2 Phase 2: Mutation and Ablation. We replay both agents against the three mutated datasets (M1: Amplification, M2: Jitter, M3: Injection) defined in Section 2.2.1. To address **RQ4**, we extend this phase with a parameter ablation study, varying intensities across three levels:

- **Amplification (k):** $\{1.25, 1.50, 2.00\}$
- **Jitter Range ($\delta_{\max}$):** $\{5, 15, 30\}$ minutes.
- **Injection Ratio (ρ):** $\{10\%, 20\%, 35\%\}$ of window volume.

This results in a total of 18 additional perturbed trajectories for sensitivity curve mapping.

3.3.3 Phase 3: Divergence and Consensus Analysis. Finally, we apply the Layer 3 metrics to the generated trajectories:

(1) **ADR Analysis:** Quantifying the per-agent sensitivity to each mutation type.
(2) **Consensus Mapping:** Measuring the drift in inter-agent agreement (C_d) across baseline and all mutated datasets.
(3) **Flip Rate Attribution:** Isolating injected windows to measure targeted decision flips and, when policy/final actions are separated, attribute whether flips were suppressed by the oracle.

3.4 Evaluation Results

3.4.1 RQ1: Agent Robustness to Mutations. Table 1 and Figure 2 quantify agent robustness via the Action Difference Ratio (ADR). Lower ADR values signify higher behavioral stability under input perturbation.

Observations. The Deterministic agent consistently exhibits higher logic drift than FinGPT, with an average ADR of 26.56%

Table 1: Action Difference Ratio (ADR %) per agent–mutation pair.

Agent	M1 (Sent.)	M2 (Jitter)	M3 (Inject.)	Mean ADR
Deterministic	21.88%	39.06%	18.75%	26.56%
FinGPT	4.69%	10.94%	3.12%	6.25%

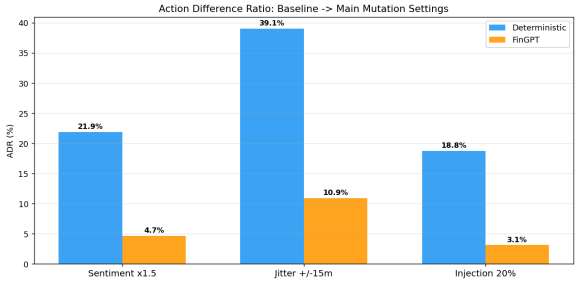

Figure 2: RQ1: Action Difference Ratio comparison across mutation types.

compared to FinGPT's 6.25%. For both agents, **temporal jitter (M2)** represents the worst-case perturbation, yielding ADRs of 39.06% and 10.94% respectively. Conversely, **adversarial injection (M3)** caused the least overall trajectory divergence, particularly for FinGPT, which remained stable in 96.88% of windows.

Root Cause Analysis. The fragility of the Deterministic agent under M2 (Jitter) is a byproduct of its **boundary-proximity sensitivity**. Since the threshold policy ($\theta^+ = 0.04, \theta^- = -0.01$) relies on per-window averages, shifting tweets across 5-minute window boundaries drastically alters the local mean. This "boundary-crossing" effect triggers frequent BUY/SELL flips, suggesting the agent lacks a smoothing mechanism to handle temporal noise.

In contrast, the apparent robustness of the FinGPT agent is primarily attributed to **output sparsity**. Analysis of the baseline trajectory reveals a "conservative bias," with the model assigning HOLD to 92.2% of windows. Because the LLM (Bloom-560m) lacks financial fine-tuning, it defaults to a low-risk fallback action, which inherently resists mutation-induced changes. While this results in a lower ADR, it indicates a lack of *signal sensitivity* rather than superior robustness. These results highlight a fundamental trade-off: rule-based systems provide high sensitivity at the cost of stability, while un-tuned LLMs provide stability at the cost of actionable intelligence.

3.4.2 RQ2: Inter-Agent Behavioral Consensus. We evaluate the decision alignment between the Deterministic and FinGPT agents to determine if input perturbations cause structural divergence in their trading logic. Table 2 and Figure 3 summarize these results.

Observations. Baseline consensus stands at 73.44%, revealing a pre-existing logic gap where agents disagree on one in four decisions. **Sentiment amplification (M1)** induces the most severe collapse in agreement, with consensus dropping to 53.12%. The primary disagreement mode is a *deterministic-BUY vs. LLM-HOLD* conflict. Notably, the risk oracle only activated during the adversarial injection (M3) phase, overriding 8 windows for each agent.

Table 2: Inter-agent consensus and oracle trigger rates across datasets.

Dataset	Windows	Agree	Consensus	Oracle (Det/FGT)
Baseline	64	47	73.44%	0 / 0
Sentiment (×1.5)	64	34	53.12%	0 / 0
Jitter (±15m)	76	52	68.42%	0 / 0
Injection (20%)	64	43	67.19%	8 / 8

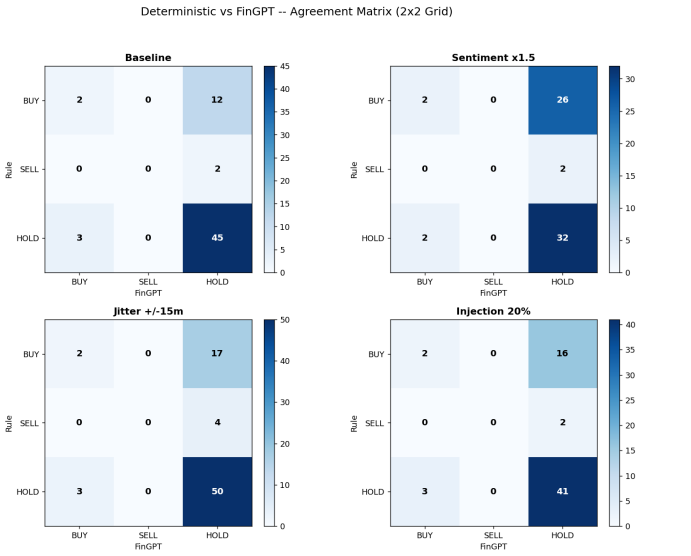

Figure 3: RQ2: 3×3 action agreement matrices. Diagonal cells represent consensus; off-diagonal cells represent logic divergence.

Root Cause Analysis. The consensus degradation under M1 is driven by **asymmetric sensitivity to signal magnitude**. As sentiment scores are amplified, a higher volume of windows crosses the Deterministic agent's low BUY threshold ($\theta^+ = 0.04$). However, the FinGPT agent exhibits **behavioral anchoring**; it remains unresponsive to increased sentiment intensity and continues to default to HOLD. This suggests that the LLM's decision logic is not linearly correlated with sentiment magnitude in the same way as a threshold-based system.

The oracle's selective activation in M3 confirms that the **conjunctive safety condition** ($\sigma_w > 0.30 \wedge r_w^- > 0.08$) effectively isolates targeted manipulation. Because the adversarial injection merges synthetic directional tweets with neutral baseline data, it creates the specific high-variance, high-conflict signature required to trigger the override—a signature that natural sentiment shifts (M1) or temporal noise (M2) do not replicate.

3.4.3 RQ3: Adversarial Susceptibility. We evaluate susceptibility to targeted manipulation by measuring the flip rate within windows receiving synthetic injections (M3). Table 3 summarizes the flip success across increasing injection ratios, while Figure 4 visualizes the distribution of these flips across the trajectory.

Table 3: Adversarial flip rates in targeted windows (16 total targets).

Agent	ρ=0.10	ρ=0.20	ρ=0.35
Deterministic	68.75% (11/16)	75.00% (12/16)	75.00% (12/16)
FinGPT	6.25% (1/16)	12.50% (2/16)	12.50% (2/16)

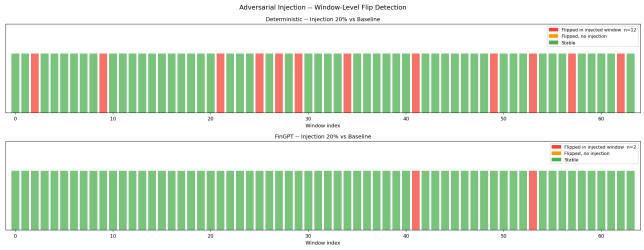

Figure 4: RQ3: Per-window flip detection (ρ=0.20). Red markers indicate successful targeted flips; orange indicates collateral drift.

Observations. In this controlled target-window setting, the Deterministic agent shows high susceptibility to adversarial injection, with the flip rate peaking at 75% for $\rho \geq 0.20$. Notably, we observe a **saturation effect**: increasing the injection ratio from 0.20 to 0.35 yields no additional flips. FinGPT remains largely unresponsive, with a maximum flip rate of 12.5%. This disparity confirms that the deterministic thresholds are significantly easier to manipulate than the LLM's latent decision logic.

Root Cause Analysis. The Deterministic agent's high flip rate is a direct consequence of **Boundary Proximity Vulnerability**. By targeting near-neutral windows ($|\bar{s}_w| \leq 0.08$), the injection exploits windows that sit mathematically closest to the agent's decision boundaries ($\theta^+ = 0.04$, $\theta^- = -0.01$). Even a low-volume injection ($n_w^{\text{syn}} \approx 3$) provides sufficient weight to shift the mean sentiment across a threshold.

The saturation observed at $\rho = 0.35$ indicates that for the 12 flipped windows, the **decision boundary crossing** is binary and achieved early; additional synthetic volume provides diminishing returns. The remaining stable windows may reflect oracle suppression, insufficient boundary crossing, or cases where the baseline action was already unchanged. To separate these mechanisms, we should report policy-level flips and final-action flips separately. Because M3 creates high-conflict signals (directional tweets injected into neutral/negative baseline noise), the conjunctive oracle condition ($\sigma_w > 0.30 \wedge r_w^- > 0.08$) successfully suppresses the flip, forcing the final action back to HOLD despite the policy's shift to BUY or SELL.

3.4.4 RQ4: Mutation Intensity Sensitivity. We evaluate the scaling behavior of logic drift by varying the intensity of each mutation operator across three levels. Table 4 and Figure 5 report the ADR sensitivity curves for both agents.

Observations. Three distinct scaling regimes emerge. For sentiment amplification, the Deterministic agent exhibits a **linear monotonic increase** in ADR, while FinGPT remains nearly invariant. For temporal jitter, both agents show heightened sensitivity, though FinGPT's response is non-monotonic, dipping at the 15-minute mark. Finally, for adversarial injection, both agents exhibit a sharp **saturation plateau** beyond $\rho = 0.20$, confirming that marginal increases in synthetic volume do not induce further logic flips.

Table 4: Ablation ADR (%) across mutation intensity parameters.

Mutation	Parameter	Det ADR (%)	FinGPT ADR (%)
Sentiment Amp.	$k = 1.25$	10.94	6.25
	$k = 1.50$	21.88	4.69
	$k = 2.00$	32.81	6.25
Temporal Jitter	$\delta_{\max} = 5$ min	32.81	15.62
	$\delta_{\max} = 15$ min	39.06	10.94
	$\delta_{\max} = 30$ min	42.19	17.19
Adversarial Inj.	$\rho = 0.10$	17.19	1.56
	$\rho = 0.20$	18.75	3.12
	$\rho = 0.35$	18.75	3.12

Root Cause Analysis. The monotonic response of the Deterministic agent to amplification stems from the linear shift in $\bar{s}_w$: increasing k pushes a progressively larger fraction of the sentiment distribution past the fixed decision thresholds. FinGPT's lack of response suggests a **granularity gap** in the base LLM; while the numeric shifts are present in the prompt, the model is not calibrated to distinguish between these magnitudes, resulting in a stable but insensitive trajectory.

The non-monotonicity of FinGPT under temporal jitter (M2) is likely a **compositional artifact** of window redistribution. Larger jitter ranges increase the total window count (e.g., 76 windows at ±15 min) by dispersing tweets across new boundaries. This alters the matched window set's composition, introducing variability that can cause non-linear ADR fluctuations in non-rule-bound models. The injection plateau validates our saturation hypothesis: once the neutral window mean is pushed across the threshold boundary at $\rho = 0.20$, additional volume is redundant, proving that small-scale, targeted injections are as effective as large-scale ones.

4 Related Work

4.1 Agentic AI and Trajectory Validation

The verification of autonomous agents often focuses on goal-reachability and safety constraints. Recent literature has shifted from evaluating static model outputs to analyzing the "trajectory" of agentic workflows—the sequence of internal states and external actions taken to achieve a task. Existing agent evaluations often inspect task success or trajectory-level behavior, but our setting focuses on whether an agent's action trajectory remains stable under controlled perturbations to the input environment.

4.2 Mutation and Metamorphic Testing

Mutation testing is a well-established software engineering technique used to evaluate the robustness of test suites by introducing

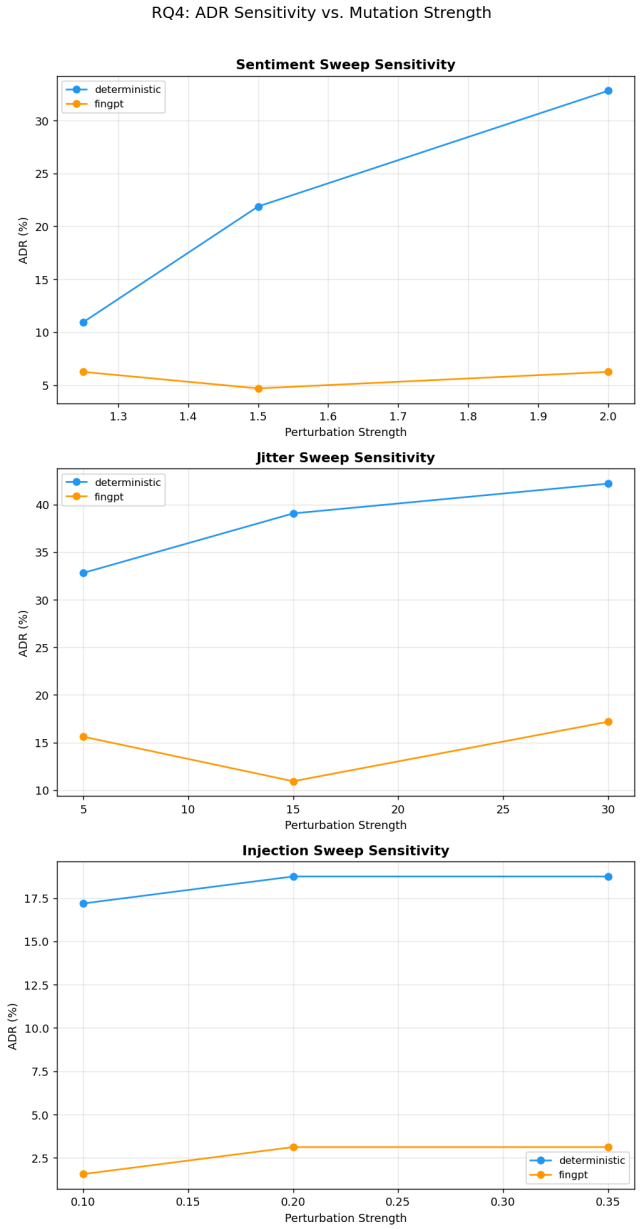

Figure 5: RQ4: ADR sensitivity curves. Each panel illustrates how drift scales with perturbation strength, highlighting distinct sensitivity profiles for each agent.

deliberate faults into source code [5]. Our research adapts this methodology to the input domain; by mutating environmental sentiment signals rather than code, we evaluate the robustness of the decision logic itself. This approach is closely related to Metamorphic Testing (MT), which addresses the "oracle problem" by defining relations between input changes and expected output shifts [3]. We utilize MT principles to validate that trading agents react consistently to scaled sentiment intensities without needing a perfect ground-truth trading oracle. For example, moderate sentiment amplification should not cause frequent polarity-inconsistent transitions such as BUY→SELL, and small temporal jitter should not induce large trajectory divergence if the decision logic is temporally robust.

4.3 Concept Drift vs. Logic Drift

While the machine learning community has extensively researched "concept drift"—where the statistical properties of a target variable change over time—the focus is typically on model accuracy and retraining [4]. Our work introduces the concept of "logic drift," which specifically targets the structural stability of the agent's software-driven decision logic. This allows us to quantify how adversarial inputs like "pump-and-dump" spikes cause an agent to violate its original risk-management profile, even when no ground-truth correctness oracle is available.

4.4 Reliability in Financial Systems

Profitability-oriented backtesting is useful for evaluating economic performance, but it does not directly test whether a decision policy is behaviorally stable under controlled perturbations. Our work complements financial evaluation by adding regression-style trajectory tests for sentiment-driven action-selection agents.

5 Threats to Validity

External Validity. Our findings are limited by the **scope of the experimental setting**. We use a 10,000-tweet subset of the Cryptocurrency Tweets Dataset focused only on BTC and ETH from October 2021 to November 2022. After filtering and 5-minute aggregation, the baseline contains only 64 coin-specific windows, so the results should be interpreted as a *controlled robustness case study* rather than a large-scale trading benchmark. Logic drift patterns may differ for lower-liquidity coins, other market periods, or different social media platforms. In addition, our LLM agent uses `BLOOM-560M` as a lightweight open-source backbone; larger or domain-specific financial LLMs may show different sensitivity profiles. Finally, our system evaluates BUY, SELL, and HOLD decision stability, but does not model price execution, portfolio state, transaction costs, or realized profit and loss.

Internal Validity. Several implementation choices may affect the measured drift. The temporal jitter and adversarial injection mutations use **fixed random seeds**, so multi-seed evaluation would provide stronger evidence that the observed ADR and flip-rate patterns are not sampling artifacts. The deterministic thresholds and shared risk-oracle parameters are manually fixed rather than tuned on a validation set, meaning some observed fragility may reflect parameter choice. Temporal jitter can also create or remove 5-minute window IDs; since ADR is computed over matched baseline–mutated windows, comparisons across jitter strengths may depend on the resulting matched-window composition. Finally, all downstream features depend on FinBERT sentiment scores, so FinBERT misclassification or calibration errors may propagate into agent decisions.

Construct Validity. We operationalize logic drift using the **Action Difference Ratio (ADR)**, which measures whether the final

discrete action changes relative to baseline. This is useful for regression testing but simplifies decision behavior. ADR treats all action changes equally, even though BUY→SELL is more severe than HOLD→BUY, and it cannot capture confidence changes within the same action label. Similarly, inter-agent consensus measures agreement rather than correctness; high consensus may result from shared HOLD behavior or oracle suppression. Therefore, logic drift in this study should be understood as *observable trajectory divergence under controlled perturbations*, not direct evidence of changes in the agent's internal reasoning process.

6 Conclusion

In this work, we introduced a mutation-testing framework for quantifying observable logic drift in sentiment-driven trading decision agents. By evaluating a deterministic rule-based policy and a financial LLM agent under sentiment amplification, temporal jitter, and adversarial tweet injection, we identified distinct fragility regimes across agent designs.

The deterministic agent was highly sensitive to temporal noise (39.06% ADR) and targeted manipulation (75% flip rate), indicating that fixed threshold policies can be vulnerable near decision boundaries. In contrast, the LLM agent exhibited lower ADR primarily because of a conservative HOLD bias, suggesting apparent stability rather than genuinely calibrated decision robustness. We also found that the shared risk oracle suppressed only a subset of adversarial cases, particularly when injected tweets created a high-conflict sentiment signature.

These results suggest that mutation-based trajectory analysis can complement profitability-oriented evaluation by exposing behavioral instability that may not be visible through backtesting alone. Future work should evaluate larger financial LLMs, incorporate price-aware paper-trading simulation, run multi-seed robustness checks, and develop weighted drift metrics that distinguish minor action changes from severe policy reversals.

References

[1] aiok. 2022. Crypto Tweets: Cryptocurrency Tweets from 2021 to 2022 Nov. Kaggle Dataset. https://www.kaggle.com/datasets/leoth9/crypto-tweets Dataset containing 10k tweets including bitcoin, ethereum, tether, and usdc.

[2] Dogu Araci. 2019. FinBERT: Financial Sentiment Analysis with Pre-trained Language Models. *arXiv preprint arXiv:1908.10063* (2019).

[3] Tsong Yueh Chen, Shing Chi Cheung, and S. M. Yiu. 1998. Metamorphic testing: a new approach for generating next test cases. *Technical Report, HKUST-CS98-01* (1998).

[4] João Gama, Indrę Žliobaitę, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. 2014. A survey on concept drift adaptation. *ACM computing surveys (CSUR)* 46, 4 (2014), 1–37.

[5] Milos Ojdanic, Ezekiel Soremekun, Renzo Degiovanni, Mike Papadakis, and Yves Le Traon. 2023. Mutation Testing in Evolving Systems: Studying the Relevance of Mutants to Code Evolution. *ACM Transactions on Software Engineering and Methodology* 32, 1 (2023), 1–39.

[6] Lavinia Rognone, Stuart Hyde, and S Sarah Zhang. 2020. News sentiment in the cryptocurrency market: An empirical comparison with forex. *International Review of Financial Analysis* 69 (2020), 101462.

[7] Teven Le Scao, Angela Fan, et al. 2022. BLOOM: A 176B-Parameter Open-Access Multilingual Language Model. *arXiv preprint arXiv:2211.05100* (2022).

[8] Jiahua Xu and Benjamin Livshits. 2019. The Anatomy of a Cryptocurrency Pump-and-Dump Scheme. In *28th USENIX Security Symposium*. 1609–1625.

[9] Hongyang Yang, Xiao-Yang Liu, and Christina Dan Wang. 2023. FinGPT: Open-Source Financial Large Language Models. *arXiv preprint arXiv:2306.06031* (2023). Presented at the FinLLM Workshop at IJCAI 2023.